Fetch API – Practice Book

This guide is written in a **practice-first, book-style format**. Each section has: - Concept - Syntax - Practice example

Read in order and type the code yourself.

Chapter 1: What is Fetch?

Concept

`fetch` is a JavaScript API used to communicate with servers using HTTP. It is used to **send requests** and **receive responses**.

Syntax

```
fetch(url, options)
```

Practice

```
fetch("https://jsonplaceholder.typicode.com/posts");
```

Chapter 2: HTTP Methods (Core Operations)

Concept

Fetch works with HTTP methods: - GET – read data - POST – create data - PUT – replace data - PATCH – update part of data - DELETE – remove data

---

Chapter 3: GET – Fetching Data

Syntax

```
fetch(url)
```

Practice

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then(res => res.json())
  .then(data => console.log(data));
```

Chapter 4: POST – Sending Data

Concept

POST is used to send data to a server.

Syntax

```
fetch(url, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(data)
})
```

Practice

```
fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "Hello", body: "Fetch practice" })
});
```

Chapter 5: PUT – Replace Entire Data

Concept

PUT replaces the **entire resource**.

Practice

```
fetch("https://jsonplaceholder.typicode.com/posts/1", {
  method: "PUT",
  headers: { "Content-Type": "application/json" },
```

```
  body: JSON.stringify({ title: "Updated", body: "New content" })
});
```

Chapter 6: PATCH – Update Partial Data

Concept

PATCH updates **only specific fields**.

Practice

```
fetch("https://jsonplaceholder.typicode.com/posts/1", {
  method: "PATCH",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "Only title updated" })
});
```

Chapter 7: DELETE – Remove Data

Concept

DELETE removes a resource by ID.

Practice

```
fetch("https://jsonplaceholder.typicode.com/posts/1", {
  method: "DELETE"
});
```

Chapter 8: Headers

Concept

Headers send metadata to the server.

Practice

```
fetch(url, {
  headers: {
    "Content-Type": "application/json",
    "Authorization": "Bearer token"
  }
});
```

Chapter 9: Response Handling

Concept

Fetch returns a **Response object**.

Practice

```
fetch(url)
  .then(res => {
    if (!res.ok) throw new Error("Failed");
    return res.json();
  })
  .then(data => console.log(data));
```

Chapter 10: Async / Await

Concept

`await` is cleaner than `.then()`.

Practice

```
async function getData() {
  const res = await fetch(url);
  const data = await res.json();
  console.log(data);
}
```

Chapter 11: Error Handling

Practice

```
try {
  const res = await fetch(url);
  if (!res.ok) throw new Error(res.status);
} catch (err) {
  console.error(err);
}
```

Chapter 12: Abort & Timeout

Practice

```
const controller = new AbortController();
setTimeout(() => controller.abort(), 3000);

fetch(url, { signal: controller.signal });
```

Chapter 13: Multiple Requests

Parallel

```
Promise.all([
  fetch(url1),
  fetch(url2)
]);
```

Sequential

```
const user = await fetch(url1).then(r => r.json());
const posts = await fetch(url2 + user.id).then(r => r.json());
```

Chapter 14: File Handling

Upload

```
const formData = new FormData();
formData.append("file", file);

fetch(url, { method: "POST", body: formData });
```

Download

```
const blob = await fetch(url).then(r => r.blob());
```

Final Summary

Fetch can:

- Perform HTTP methods
- Send data
- Receive data
- Handle responses
- Handle errors
- Control async flow
- Upload & download files

Final Rule

> Fetch is the **foundation of client-server communication in JavaScript**.

End of Practice Book